内建函数

>>> dir(__builtins__)

		Built-in Functions		
abs()	dict()	help()	min()	setattr()
all()	dir()	hex()	next()	slice()
any()	divmod()	id()	object()	sorted()
ascii()	enumerate()	input()	oct()	staticmethod()
bin()	eval()	int()	open()	str()
bool()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	
delattr()	hash()	memoryview()	set()	

表达式

用运算符（除了赋值运算符）连接各种类型数据的式子就是表达式

- 运算符有优先级顺序；
- 表达式必须有运算结果；
- 单个常量或变量可以看作是最简单的表达式，例如：3，‘a’



赋值

- 增量赋值操作符

+= -= *= /= %= **= <<= >>= &= ^= |=

- 链式赋值

a=b=c

- 多重赋值
  - 等号左右两边都以元组的方式出现

x, y = y, x

temp = 3.14159, 3

PI, r = temp

基本运算

- 成员运算符
  - 通过成员运算符 ‘in’ 和 ‘not in’ ，我们可以确认一个值是否是另一个值的成员。

```
>>>'me' in 'disappointment'
True
```

- 身份运算符
  - 通过身份运算符 ‘is’ 和 ‘is not’ ，我们可以确认两个值是否相同。

```
>>>10 is '10'
False
>>>10 is 10
True
```


类型转换与类型判断

内置函数bin()、oct()、hex()用来将整数转换为二进制、八进制和十六进制形式，这三个函数都要求参数必须为整数。

```
>>> bin(555)                    #把数字转换为二进制串，0b开头
'0b1000101011'
>>> oct(555)                    #转换为八进制串，0o开头
'0o1053'
>>> hex(555)                    #转换为十六进制串，0x开头
'0x22b'
```

类型转换与类型判断

```
>>> int('0x22b', 16)                    #把十六进制数转换为十进制数
555
>>> int(bin(54321), 2)                  #二进制与十进制之间的转换
54321
>>> int('0b111')                        #非十进制字符串进，必须指定第二个参数
ValueError: invalid literal for int() with base 10: '0b111'
>>> int('0b111', 0)                     #第二个参数0表示使用字符串隐含的进制
7
>>> int('0b111', 6)                     #第二个参数应与隐含的进制一致
ValueError: invalid literal for int() with base 6: '0b111'
>>> int('0b111', 2)
7
>>> int('111', 6)                       #字符串没有隐含进制
43                                      #第二个参数可以为2-36之间的数字
```


类型转换与类型判断

ord()和chr()是一对功能相反的函数，ord()用来返回单个字符的Unicode码，而chr()则用来返回Unicode编码对应的字符，str()则直接将其任意类型参数转换为字符串。

```
>>> ord('a')            #查看指定字符的Unicode编码
97
>>> chr(65)             #返回数字65对应的字符
'A'
>>> chr(ord('A')+1)     #Python不允许字符串和数字之间的加法操作
'B'
>>> chr(ord('国')+1)    #支持中文
'图'
>>> ord('董')           #这个用法仅适用于Python 3.x
33891
```

注意：只能单个字符进行处理！

类型转换与类型判断

内置函数type()和isinstance()可以用来判断数据类型，常用来对函数参数进行检查，可以避免错误的参数类型导致函数崩溃或返回意料之外的结果。

```
>>> type(3)                                  #查看3的类型
<class 'int'>
>>> type([3])                                #查看[3]的类型
<class 'list'>
>>> type({3}) in (list, tuple, dict)         #判断{3}是否为list,tuple或dict类型的实例
False
>>> isinstance(3, int)                       #判断3是否为int类型的实例
True
>>> isinstance(3j, int)
False
>>> isinstance(3j, (int, float, complex))    #判断3是否为int,float或complex类型
True
```


类型转换总结

函数名	功能	英文全称
bin(num)	二进制转换	Binary
oct(num)	八进制转换	Octal
hex(num)	十六进制转换	Hexadecimal
int(num,base)	整型(十进制)转换	Integer
ord(c)	单个字符转Unicode编码	Ordinal
chr(num)	Unicode编码转字符	Character
str()	任意类型参数转换为字符串	String
type()	查看数据类型	Type
isinstance()	判断数据类型	Is Instance

eval() 函数

- 评估函数，用来执行一个字符串表达式，并返回表达式的值。

```
>>>x = 7
>>> eval( '3 * x' )
21
```

相当于去掉字符串的引号，对表达式进行计算并输出结果。

最值与求和

函数max()和min()还支持default参数和key参数，其中default参数用来指定可迭代对象为空时默认返回的最大值或最小值，而key参数用来指定比较大小的依据或规则，可以是函数或lambda表达式。

```
>>> max(['2', '111'])                    #不指定排序规则
'2'
>>> max(['2', '111'], key=len)           #返回最长的字符串
'111'
>>> print(max([], default=None))          #对空列表求最大值，返回空值None
None
```

排序

sorted()对列表、元组、字典、集合或其他可迭代对象进行排序并返回新列表。

```
>>> x=[2, 4, 0, 6, 10, 7, 8, 3, 9, 1, 5]
>>> sorted(x)                                    #以默认规则排序
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> sorted(x, key=str)                          #按转换成字符串以后的大小升序排列
[0, 1, 10, 2, 3, 4, 5, 6, 7, 8, 9]
>>> sorted(x, reverse=True)       #按逆序（从大到小）顺序排列
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```


逆序

reversed()对可迭代对象（生成器对象和具有惰性求值特性的zip、map、filter、enumerate等类似对象除外）进行翻转（首尾交换）并返回可迭代的reversed对象。

reversed(iterable) -> reversed object

```
>>> x=[2, 4, 0, 6, 10, 7, 8, 3, 9, 1, 5]

>>> reversed(x)                                 #逆序，返回reversed对象
<list_reverseiterator object at 0x0000000003089E48>

>>> list(reversed(x))                           #reversed对象是可迭代的
[5, 1, 9, 3, 8, 7, 10, 6, 0, 4, 2]
```

惰性求值特性

- 惰性求值特性: 其中的元素只能使用一次，不支持用内置函数len()计算元素个数，也不支持使用内置函数reversed()再次翻转。
- reversed、zip、map、filter、enumerate等都具有惰性求值的特性。

```
>>> x=[2, 4, 0, 6, 10, 7, 8, 3, 9, 1, 5]
>>> y = reversed(x)
>>> 2 in y
True
>>> 2 in y
False
```


枚举与迭代

enumerate()函数用来枚举可迭代对象中的元素，返回可迭代的enumerate对象，其中每个元素都是包含索引和值的元组。

```
>>> list(enumerate('abcd'))                          #枚举字符串中的元素
[(0, 'a'), (1, 'b'), (2, 'c'), (3, 'd')]

>>> list(enumerate(['Python', 'Greate']))            #枚举列表中的元素
[(0, 'Python'), (1, 'Greate')]

>>> list(enumerate({'a':97, 'b':98, 'c':99}.items())) #枚举字典中的元素
[(0, ('c', 99)), (1, ('a', 97)), (2, ('b', 98))]
```

zip()函数

zip()函数用来把多个可迭代对象中的元素压缩到一起，返回一个可迭代的zip对象，其中每个元素都是包含原来的多个可迭代对象对应位置上元素的元组，如同拉拉链一样。

```
>>> list(zip('abcd', [1, 2, 3]))              #压缩字符串和列表
[('a', 1), ('b', 2), ('c', 3)]
>>> list(zip('123', 'abc', ',.!'))            #压缩3个序列
[('1', 'a', ','), ('2', 'b', '.'), ('3', 'c', '!')]
>>> x = zip('abcd', '1234')
>>> list(x)
[('a', '1'), ('b', '2'), ('c', '3'), ('d', '4')]
```



zip()函数

注意：zip()函数具有惰性求值的特点，zip对象中的每个元素都只能使用一次，访问过的元素无法再次访问，并且只能从前往后逐个访问zip对象中的元素。

```
>>> x = zip('abcd', '1234')
>>> list(x)
[('a', '1'), ('b', '2'), ('c', '3'), ('d', '4')]
>>> tuple(x)
()                          #第二次访问为空
>>> x = zip('abcd', '1234')
>>> list(zip(*x))   #zip(*seq)为解压缩操作
[('a', 'b', 'c', 'd'), ('1', '2', '3', '4')]
```



map()函数

- 内置函数map()把一个函数依次作用于可迭代对象的每个元素上，并返回一个可迭代的map对象作为结果。 map对象中每个元素是原序列中元素经过函数func处理后的结果，map()函数不对原始数据做任何修改。
- map(func, *iterables) -> map object

```
>>>list(map(str,range(3)))
['0', '1', '2']
```


filter()函数

- 内置函数filter()用于使用指定函数描述的规则对序列中的元素进行过滤。

  filter(func or None, iterable) -> filter object

```
>>>seq=['abcd','1234','.!+','']
>>>list(filter(str.isdigit, seq))
['1234']
```


python

```
str.isdigit()   # 是否全部是数字 '123'→True，'12a'→False
str.isalpha()   # 是否全部是字母 'abc'→True
str.isalnum()   # 是否字母+数字，不含符号 'a12'→True
str.isspace()   # 是否全是空白（空格、制表符、换行）
str.islower()   # 是否全小写字母
str.isupper()   # 是否全大写字母
str.istitle()   # 是否每个单词首字母大写，如 'Hello World'
```

列表方法

append()
copy()
count()
extend()
index()
insert()
pop()
remove()
reverse()
sort()

参数的作用：

list_name.sort(key=None, reverse=False)

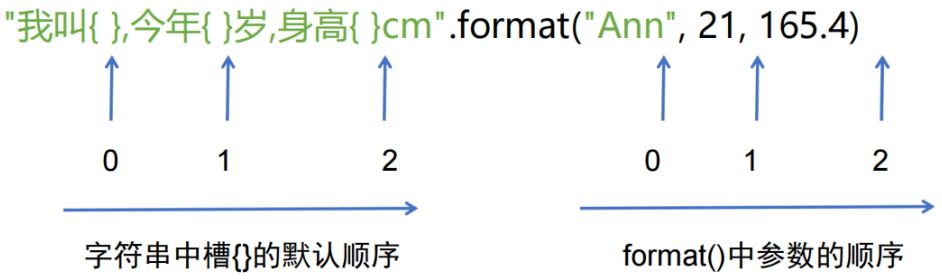
```
>>> numList= [3, 11, 5, 8, 16, 1]
>>> fruitList= ['apple', 'banana', 'pear', 'lemon', 'avocado']
>>> numList.sort(reverse = True)
>>> numList
[16, 11, 8, 5, 3, 1]
>>> fruitList.sort(key = len)
>>> fruitList
['pear', 'apple', 'lemon', 'banana', 'avocado']
```


字符串常用方法

capitalize()	center()	count()	encode()	endswith()	find()
format()	index()	isalnum()	isalpha()	isdigit()	islower()
isspace()	istitle()	isupper()	join()	ljust()	lower()
lstrip()	maketrans()	partition()	replace()	rfind()	rindex()
rjust()	rpartition()	rstrip()	split()	splitlines()	startswith()
strip()	swapcase()	title()	translate()	upper()	zfill()

用format函数格式化字符串

- <模板字符串>.format(<逗号分隔的参数>)



用format函数格式化字符串

- 槽内部对格式化的配置方式 {<参数序号>: <格式控制标记>}

"我叫{2},今年{1:*>5d}岁,身高{0:6.2f}cm".format(165.4, 21, "Ann")

输出结果：'我叫Ann,今年***21岁,身高165.40cm'

:	<填充>	<对齐>	<宽度>	<,>	<.精度>	<类型>
引导符号	用于填充的单个字符	< 左对齐 > 右对齐 ^ 居中对齐	槽设定的输出宽度	数字的千位分隔符	浮点数小数精度 或 字符串最大输出长度	整数类型 b, c, d, o, x, X 浮点数类型 e, E, f, %

其他常用格式说明符

符号	描述
+m.nf	输出带符号（若正整数输出“+”号）的数，保留n位小数，整个输出占m列（若实际宽度超过m则突破m的限制）
<	左对齐，默认用空格填充右边
0>5d	右对齐，用0填充左边，宽度为5
^	居中对齐
{{}}	输出一个{}

[对齐说明符][符号说明符][最小宽度说明符][.精度说明符][类型说明符]

```
>>>age,height=21,1.758
>>>print("Age:{0:<5d},Height:{1:5.2f}".format(age,height))
Age:21,Height:1.76
```


转义字符

转义字符	含义	转义字符	含义
\b	退格，把光标移动到前一列位置	\\	一个斜线\
\f	换页符	\’	单引号’
\n	换行符	\”	双引号”
\r	回车	\ooo	3位八进制数对应的字符
\t	水平制表符	\xhh	2位十六进制数对应的字符
\v	垂直制表符	\uhhhh	4位十六进制数表示的Unicode字符

```
>>> aStr= '\101\t\x41\n'
>>> bStr= '\141\t\x61\n'
>>> print(aStr, bStr)
A    A
A    a
```

转义字符

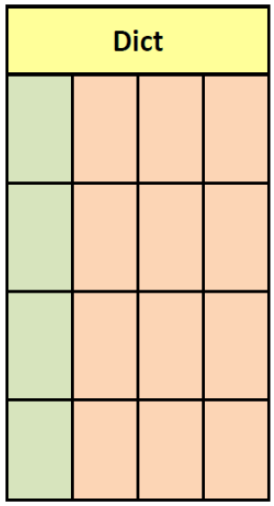
- 如果想让转义字符在字符串中不起作用应该如何做呢?

```
>>> aStr= '\101\t\x41\n'
>>> bStr= '\141\t\x61\n'
>>> print(aStr, bStr)
A    A
A    a

>>> aStr= r'\101\t\x41\n'
>>> bStr= r'\141\t\x61\n'
>>> print(aStr, bStr)
\101\t\x41\n \141\t\x61\n
```

串前加上r这个字符

字典



- 一种映射类型
  - 键（key）
  - 值（value）
  - key-value对
- “键”不允许重复
- “键”可以是Python中任意不可变数据
- 无序可变序列
- 采用大括号{}和dict()创建，键值对用冒号：表示

创建字典

- 直接创建
- 利用dict函数

Source

```
>>> aInfo = {'Ann': 3000, 'John':2000, 'Lucy':4500, 'Jessica':8000}
>>> info = [('Ann',3000), ('John',2000), ('Lucy',4500), ('Jessica',8000)]
>>> bInfo = dict(info)
>>> cInfo= dict([['Ann',3000], ['John',2000], ['Lucy',4500], ['Jessica',8000]])
>>> dInfo= dict(Ann=3000, John=2000, Lucy=4500, Jessica=8000)
{'Jessica': 8000, 'Ann': 3000, 'Lucy': 4500, 'John': 2000}
```



创建字典

- **问题解决**：创建员工信息表时如何将所有员工的工资默认值设置为3000?

Source

```
>>> aDict= {}.fromkeys(('Ann', 'John', 'Lucy', 'Jessica'),3000)
>>> aDict
{'Ann': 3000, 'John': 3000, 'Lucy': 3000, 'Jessica': 3000}
>>>sorted(aDict)
['Ann', 'Jessica', 'John', 'Lucy']
```

生成字典

- **问题解决：**已知有姓名列表和工资列表，如何生成字典类型的员工信息表？

Source

```
>>> names = ['Ann', 'John', 'Lucy', 'Jessica']
>>> salaries = [3000, 2000, 4500, 8000]
>>> dict(zip(names,salaries))
{'Ann': 3000, 'John': 2000, 'Lucy': 4500, 'Jessica': 8000}
```


字典的基本操作

Source

```
>>> aInfo = {'Ann': 3000, 'John': 2000, 'Lucy': 4500, 'Jessica': 8000}
>>> aInfo['John']                    键值查找
2000
>>> aInfo['Jessica'] = 9999          更新
>>> aInfo
{'Ann': 3000, 'John': 2000, 'Lucy': 4500, 'Jessica': 9999}
>>> aInfo['Justin'] = 1000           添加
>>> aInfo
{'Ann': 3000, 'John': 2000, 'Lucy': 4500, 'Jessica': 9999, 'Justin': 1000}
>>>  'Tim' in aInfo                   成员判断
False
>>> del aInfo['Justin']              删除字典成员
>>> aInfo
{'Ann': 3000, 'John': 2000, 'Lucy': 4500, 'Jessica': 9999}
```

字典方法

- **问题解决**：已知有员工姓名和工资信息表{'Ann': 3000, 'John': 2000, 'Lucy': 4500, 'Jessica': 8000}，如何单独输出员工姓名和工资金额?

Source

```
>>> aInfo = {'Ann': 3000, 'John': 2000, 'Lucy': 4500, 'Jessica': 8000}
>>> aInfo.keys()              #字典的键
['Ann', 'John', 'Lucy', 'Jessica']
>>> aInfo.values()            #字典的值
[3000, 2000, 4500, 8000]
>>> for k, v in aInfo.items():   #字典中的键、值对
    print(k, v)
```


字典方法

- **问题解决：**人事部门有两份人员和工资信息表，第一份是原有信息，第二份是公司中有工资更改人员和新进人员的信息，如何处理可以较快地获得完整的信息表?

Source

```
>>> aInfo = {'Ann': 3000, 'John': 2000, 'Lucy': 4500, 'Jessica': 8000}
>>> bInfo = {'Ann': 4000, 'John': 5000, 'Lucy': 4500, 'Tom': 8999}
>>> aInfo.update(bInfo)
>>> aInfo
{'Ann': 4000, 'John': 5000, 'Lucy': 4500, 'Jessica': 8000, 'Tom': 8999}
```

字典方法

- 删除字典

```
>>> aStock = {'AXP': 78.51, 'BA':184.76}
>>> bStock= aStock
>>> aStock = {}
>>> bStock
{'BA': 184.76, 'AXP': 78.51}
```

```
>>> aStock = {'AXP': 78.51, 'BA': 184.76}
>>> bStock= aStock
>>> aStock.clear()
>>> aStock
{}
>>> bStock
{}
```

clear()	copy()	fromkeys()	get()	items()
keys()	pop()	setdefault()	update()	values()

集合关系运算

数学符号	Python符号
∈	in
∉	not in
=	==
≠	!=
⊂	<
⊆	<=
⊃	>
⊇	>=

标准类型运算符

数学符号	Python符号
∩	&
∪	|
- 或 \	-
Δ	^

集合类型运算符

运算符可复合

&= |= -= ^=

注意：关系运算符>,>=,<,<=作用于集合时表示集合之间的包含关系，而不是比较集合中元素大小关系。

集合方法

面向所有集合

s.issubset(t)
issuperset(t)
union(t)
intersection(t)
difference(t)
symmetric_difference(t)
copy()

面向可变集合

update(t)
intersection_update(t)
difference_update(t)
symmetric_difference_update(t)
add(obj)
remove(obj)
discard(obj)
pop()
clear()

异常

- 查看异常类

```
>>> dir(__builtins__)
```

类　名	描　述
BaseException	所有异常的基类
Exception	常规异常的基类
AttributeError	对象不存在此属性
IndexError	序列中无此索引
IOError	输入/输出操作失败
KeyboardInterrupt	用户中断执行(通常输入Ctr-C)
KeyError	映射中不存在此键
NameError	找不到名字（变量）
SyntaxError	Python 语法错误
TypeError	对类型无效的操作
ValueError	传入无效的参数
ZeroDivisionError	除(或取模)运算的第二个参数为0

异常处理结构

- try...except...

```
try:
    #可能会引发异常的代码，先执行一下试试
except Exception[ as reason]:
    #如果try中的代码抛出异常并被except捕捉，就执行这里的代码
```

```
#exception2.py
try:
    num1 = int(input('Enter the first number: '))
    num2 = int(input('Enter the second number: '))
    print(num1 / num2)
except ValueError:
    print('Please input a digit!')
```

支持多个except子句；
支持一个except块捕捉多个异常

异常处理结构

- try...except...else...

```
try:
    #可能会引发异常的代码
except Exception [ as reason]:
    #用来处理异常的代码
else:
    #如果try子句中的代码没有引发异常，就继续执行这里的代码
```

```
# Filename: exception3.py
try:
    num1 = int(input('Enter the first number: '))
    num2 = int(input('Enter the second number: '))
    print(num1 / num2)
except(ValueError, ZeroDivisionError):
    print('Invalidinput!')
else:
    print('Aha, everything is OK.')
```

异常处理结构

- try...except...finally...

```
try:
    #可能会引发异常的代码
except Exception [ as reason]:
    #处理异常的代码
finally:
    #无论try子句中的代码是否引发异常，都会执行这里的代码
```

```
# Filename: exception4.py
def finallyTest():
    try:
        x = int(input('Enter the first number: '))
        y = int(input('Enter the second number: '))
        print(x / y)
        return 1
    except Exception as err:
        print(err)
        return 0
    finally:
        print('It is a finally clause.')
result = finallyTest()
print(result)
```


一些迭代工具

内建的zip函数就可以直接用来进行并行迭代，可以把两个序列“压缩”在一起，然后返回一个元组的对象：

zip(names, ages)

zip(iter1 [,iter2 [...]]) --> zip object

现在我们可以在循环中解包元组：

```
>>>for name, age in zip(names, ages):
        print(name, 'is', age, 'years old')
```

Ann is 12 years old

Nina is 20 years old

George is 102 years old

zip可以处理不等长序列。
解压缩的方式是zip(*zipped)

一些迭代工具

2，按索引迭代

有些时候想要迭代访问序列中的元素，同时还要获取当前元素的索引。

例如在一个列表中替换所有包含'xxx'的子元素。

可以使用内建的enumerate函数：

```
for index, string in enumerate(list1):
    if 'xxx' in list1:
      list1[index]= '[censored]'
```

这个函数可以在提供索引的地方迭代索引-值对。

循环中的else子句

else子句仅在没有调用break时执行，可以把它看作是完整执行完循环的“奖励”。**问题解决**：寻找（99,81）区间内的平方数

```
#filename:elsedemo.py
from math import sqrt
for n in range(99,81,-1):
     root=sqrt(n)
     if root==int(root):
       print(n)
       break
else:
    print("Didn't find it!")

```

Output: Didn't find it!

列表推导式（列表解析）

- 列表推导式在逻辑上等价于一个循环语句，只是形式上更加简洁。例如：

```
>>> aList = [x*x for x in range(10)]
```

相当于

```
>>> aList = []
>>> for x in range(10):
        aList.append(x*x)
```


列表推导式（列表解析）

```
>>> [x for x in range(10)]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> [x ** 2 for x in range(10)]
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> [x ** 2 for x in range(10) if x ** 2 < 50]
[0, 1, 4, 9, 16, 25, 36, 49]
>>> [(x+1, y+1) for x in range(2) for y in range(2)]
[(1, 1), (1, 2), (2, 1), (2, 2)]
```




```
[ expression for expr in sequence1
              for expr2 in sequence2 ...
              for exprN in sequenceN
              if condition ]
```

动态创建列表
简单灵活有用

__name__属性的作用

- 通过python程序的__name__属性可以识别程序的使用方式，每个python脚本在运行时都会有一个__name__属性，如果脚本作为模块被导入，则其__name__属性的值被自动设置为模块名；如果脚本作为程序直接运行，则其__name__属性值被自动设置为字符串'__main__'。

```
#hello.py
def main():
    if __name__=='__main__':
        print('这程序是直接运行的。')
    elif __name__=='hello':
        print('这程序做为模块被导入。')

main() #调用上面定义的函数
```


位置参数

- 必须以正确的顺序传入函数，调用时的数量必须和声明时的一样；
- 最常用和常见的参数形式。

```
>>> def f(x , y):
    '''x and y both correct words or not '''
    if y:
        print(x, 'and y both correct ' )
    print(x, 'is OK')
>>> f(68, False)
68 is OK
```

关键字参数

- 关键字参数是让调用者通过使用参数名区分参数;
- 允许在函数调用时改变参数列表中的参数顺序 。与函数声明无关!
- 关键字参数的使用必须要在位置参数之后!

```
>>> def f(x , y):
        '''x and y both correct words or not '''
        if y:
            print(x, 'and y both correct')
        print(x, 'is OK')
>>> f(68, False)
68 is OK
>>> f(y = False, x = 68)
68 is OK
>>> f(y = False, 68)
SyntaxError: non-keyword argafter keyword arg
>>> f(x = 68, False)
SyntaxError: non-keyword argafter keyword arg
```


默认参数（一）

- 函数的参数可以有一个默认值，如果提供有默认值，在函数定义中，默认参数以赋值语句的形式提供，并且只会被初始化一次。

```
>>> def f(x = True):
        '''whether x is a correct word or not'''
        if x:
            print('x is a correct word')
        print('OK')
>>> f()
x is a correct word
OK
>>> f(False)
OK
```

默认参数（二）

- 默认参数的值可以更改；
- 默认参数一般需要放置在参数列表的最后；
- 可以使用“函数名.__defaults__”查看默认参数的值。

```
def f(y = True, x):
    '''x and y both correct words or not'''
    if y:
        print(x, 'and y both correct')
    print(x, 'is OK')
SyntaxError: non-default argument follows default argument
```


默认参数（三）

- 多次调用函数并且不为默认值参数传递值时，默认值参数只在定义时进行一次解释和初始化，对于列表、字典这样可变类型的默认值参数，这一点可能会导致很严重的逻辑错误。例如：

```
>>> def demo(newitem, old_list=[]):
        old_list.append(newitem)
        return old_list
```

```
>>> print(demo('5', [1, 2, 3, 4]))
[1, 2, 3, 4, '5']
>>> print(demo('aaa', ['a', 'b']))
['a', 'b', 'aaa']
>>> print(demo('a'))
['a']
>>> print(demo('b')) #注意这里的输出结果
['a', 'b']
```

默认参数（四）

- 一般来说，要避免使用列表、字典、集合或其他可变序列作为函数参数默认值，对于上面的函数，更建议使用下面的写法。



```
def demo(newitem, old_list=None):
    if old_list is None:
        old_list = []
    old_list.append(newitem)
    return old_list
```


可变长度参数

- 函数定义时可以设计可变数量参数，既不确定参数总数量
  - 可变长度参数主要有两种形式：在参数名前加1个*或2个**
  - *parameter用来接受多个位置参数并将其放在一个元组中
  - **parameter接受多个关键参数并存放到字典中

```
>>> def demo(*p):
        print(p)

>>> demo(1,2,3)
(1, 2, 3)
>>> demo(1,2)
(1, 2)
```

```
>>> def demo(**p):
        for item in p.items():
            print(item)

>>> demo(x=1,y=2,z=3)
('y', 2)
('x', 1)
('z', 3)
```

传递参数时的序列解包

- 传递参数时，可以通过在实参序列前加一个星号将其解包，然后传递给多个单变量形参，等价于位置参数。

```
>>> def demo(a, b, c):
        print(a+b+c)

>>> seq = [1, 2, 3]
>>> demo(*seq)
6
>>> tup = (1, 2, 3)
>>> demo(*tup)
6
```

```
>>> dic = {1:'a', 2:'b', 3:'c'}
>>> demo(*dic)
6
>>> Set = {1, 2, 3}
>>> demo(*Set)
6
>>> demo(*dic.values())
abc
```


传递参数时的序列解包

- 如果函数实参是字典，可以在前面加两个星号进行解包，等价于关键字参数。

```
>>> def demo(a, b, c):
        print(a+b+c)
>>> dic = {'a':1, 'b':2, 'c':3}
>>> demo(**dic)
6
>>> demo(a=1, b=2, c=3)
6
>>> demo(*dic.values())
6
```

传递参数时的序列解包

- 注意：调用函数时对实参序列使用一个星号*进行解包后的实参将会被当做普通位置参数对待，并且会在关键字参数和使用两个星号**进行序列解包的参数之前进行处理。

```
>>> demo(*(1, 2, 3))        #调用，序列解包
1 2 3
>>> demo(1, *(2, 3))        #位置参数和序列解包同时使用
1 2 3
>>> demo(1, *(2,), 3)
1 2 3
>>> demo(a=1, *(2, 3))  #序列解包相当于位置参数，优先处理
Traceback (most recent call last):
  File "<pyshell#26>", line 1, in <module>
    demo(a=1, *(2, 3))
TypeError: demo() got multiple values for argument 'a'
>>> demo(c=1, *(2, 3))
2 3 1
```


传递参数时的序列解包

```
>>> demo(**{'a':1, 'b':2}, *(3,)) #序列解包不能在关键参数解包之后
SyntaxError: iterable argument unpacking follows keyword
argument unpacking

>>> demo(*(3,), **{'a':1, 'b':2})
Traceback (most recent call last):
  File "<pyshell#30>", line 1, in <module>
    demo(*(3,), **{'a':1, 'b':2})
TypeError: demo() got multiple values for argument 'a'
>>> demo(*(3,), **{'c':1, 'b':2})
3 2 1
```

lambda函数

- 匿名函数
  - 只可以包含一个表达式，该表达式的计算结果可以看作是函数的返回值，不允许包含复合语句，但在表达式中可以调用其他函数。

```
>>> def addMe2Me(x):
        'apply operation + to argument'
        return x+x
>>> addMe2Me(5)
10
>>>addMe2Me('a')
'aa'
```

```
>>> r = lambda x : x + x
>>> r(5)
10
>>>r('a')
'aa'
```


lambda函数

```
def my_add(x, y) : return x + y
```

```
lambda      x, y   :   x + y
```

```
my_add = lambda x, y :   x + y
```

```
>>> my_add(3, 5)
8
```

lambda函数的应用

- lambda函数主要用作一些特定函数或方法的参数

```
map(lambda x:x**2, range(10))
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

sorted('1a2B3Cd4', key=lambda i:(i.isupper(), i.isdigit(), i.islower()))
['a', 'd', '1', '2', '3', '4', 'B', 'C']
```

一般情况，建议使用def定义的普通参数

同名变量

- 全局变量和局部变量用同一个名字

```
>>>a = 3
>>>def f():
        a = 5
        print(a ** 2)
```

函数内部为局部变量；
同名时，局部变量会
屏蔽全局变量

global语句

- global语句强调全局变量

```
>>>def f(x):
        global a
        print(a)
        a = 5
        print(a + x)
>>>a = 3
>>>f(8)
print(a)
```

Output:
3
13
5

文件的打开

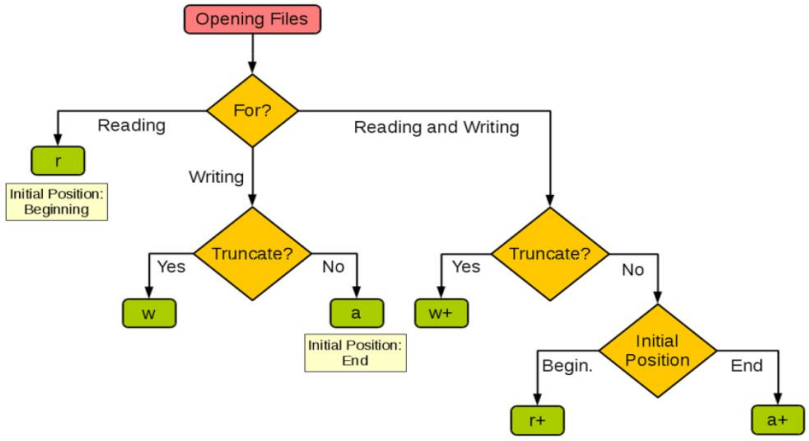
- **open(file, mode='r', buffering=1, encoding=None, errors=None,newline=None, closefd=True, opener=None)**
  - file参数指定了被打开的文件名称。
  - mode参数指定了打开文件后的处理方式。
  - buffering参数指定了读写文件的缓存模式。0表示不缓存，1表示缓存，如大于1则表示缓冲区的大小。默认值是缓存模式。
  - encoding参数指定对文本进行编码和解码的方式，只适用于文本模式，可以使用Python支持的任何格式，如GBK、utf8、CP936等等。

```
>>> f1 = open('d:\\infile.txt')
>>> f2 = open(r'd:\outfile.txt','w')
>>> f3 = open('record.dat', 'wb', 0)
```

open()函数-mode

Mode	Function
r	以读模式打开
w	以写模式打开（清空原内容）
x	以写模式打开，若文件已存在则失败
a	以追加模式打开（从EOF开始，必要时创建新文件）
r+	以读写模式打开
w+	以读写模式打开（清空原内容）
a+	以读和追加模式打开
rb	以二进制读模式打开
wb	以二进制写模式打开（参见w）
ab	以二进制追加模式打开（参见a）
rb+	以二进制读写模式打开（参见r+）
wb+	以二进制读写模式打开（参见w+）
ab+	以二进制读写模式打开（参见a+）

打开模式的总结



文件相关函数

返回值

- open()函数返回一个文件（file）对象
- 文件对象可迭代
- 有关闭和读写文件相关的函数/方法
  - f.read(), f.write(), f.readline(), f.readlines(), f.writelines()
  - f.close()
  - f.seek()

写文件-f.write()

- **file_obj.write(str)**
  - 将一个字符串写入文件

```
>>>f = open('firstpro.txt', 'w')
>>> f.write('Hello, World!')
>>> f.close()
```

firstpro.txt:
Hello, World!

```
>>>with open('firstpro.txt', 'w') as f:
    f.write('Hello, World!')
```

上下文管理器（Context Manager）with语句

- 使用with自动关闭资源，可以在代码块执行完毕后还原进入该代码块时的现场。
- 不论何种原因跳出with块，不论是否发生异常，总能保证文件被正确关闭，资源被正确释放。

```
# Filename: exception5.py
try:
    f = open('data.txt')
    for line in f:
        print(line, end = '')
except IOError:
    print('Can not open the file!')
finally:
    f.close()
```

```
# Filename: exception6.py
with open('data.txt') as f:
    for line in f:
        print(line, end='')
```


读文件-f.read()

- **file_obj.read(size=-1)**
  - 从文件中至多读出size字节数据，返回一个字符串
  - size默认值为-1，读文件直到文件结束，返回一个字符串

```
>>> with open('firstpro.txt') as f:
    p1 = f.read(5)
    p2 = f.read()
    print(p1,p2)
```

Output：
Hello, World!

其他读写函数



```
# Filename: companies_a.py
with open('companies.txt') as f:
    cNames= f.readlines()
    print(cNames)
```

- file_obj.readlines()
- file_obj.readline()
- file_obj.writelines()

Output：
['GOOGLE Inc.\n', 'Microsoft Corporation\n', 'Apple Inc.\n', 'Facebook Inc.']

其他文件相关函数

- file_obj.seek(offset , whence=0)
  - 在文件中移动文件指针，从*whence*（0表示文件头部，1表示当前位置，2表示文件尾部）偏移*offset*个字节
  - whence参数可选，默认值为0



```
# Filename: companies_b.py
s = 'Tencent Technology Company Limited'
with open('companies.txt' , 'a+') as f:
    f.writelines('\n')
    f.writelines(s)
    cNames= f.readlines()
print(cNames)
```